UQA Technical Overview

Jaepil Jeong
Cognica, Inc.
Email: jaepil@cognica.io
Date: Mar 16, 2026

"Logic pervades the world: the limits of the world are also its limits."
— Ludwig Wittgenstein

This document provides a detailed technical exposition of the Unified Query Algebra, connecting the formal theory developed across four research papers to the concrete implementation in the UQA codebase.

UQA Technical Overview
- 1. Posting Lists as Universal Abstraction
  - 1.1 The Core Thesis
  - 1.2 Boolean Algebra
  - 1.3 Implementation
- 2. Cross-Paradigm Operators
  - 2.1 Primitive Operators
  - 2.2 Operator Monoid
  - 2.3 Boolean Operators
- 3. Scoring: From BM25 to Bayesian BM25
  - 3.1 Classical BM25
  - 3.2 The Probabilistic Relevance Gap
  - 3.3 Bayesian BM25
  - 3.4 Vector Similarity and Calibration
  - 3.5 Calibration Metrics
  - 3.6 Online Parameter Learning
  - 3.7 External Prior Scoring
  - 3.8 Multi-Field Bayesian BM25
  - 3.9 Sparse Thresholding (ReLU-as-MAP)
- 4. Fusion: Combining Cross-Paradigm Signals
  - 4.1 The Conjunction Shrinkage Problem
  - 4.2 Log-Odds Conjunction
  - 4.3 Probabilistic Boolean Operations
  - 4.4 Implementation
  - 4.5 Attention-Weighted Fusion
  - 4.6 Learned-Weight Fusion
  - 4.7 Multi-Signal WAND Pruning
  - 4.8 Multi-Stage Retrieval Pipeline
  - 4.9 Fusion Gating Mechanisms
  - 4.10 Progressive Fusion
  - 4.11 Deep Learning Training Pipeline
    - 4.11.1 Analytical Parameter Estimation
    - 4.11.2 Product of Experts (PoE) Ensemble
    - 4.11.3 SQL Interface
    - 4.11.4 GPU Acceleration
    - 4.11.5 Self-Attention Layer
    - 4.11.6 Neural Network Pruning
    - 4.11.7 Model Catalog
- 5. Graph Extension

5.1 Graph-Posting List Isomorphism
5.2 Graph Operators
5.3 Subgraph Isomorphism: Complexity and Mitigations
5.4 Regular Path Expressions
5.5 openCypher Integration
5.6 Path Index Integration
5.6.1 Subgraph Indexing
5.7.1 Incremental Pattern Matching
5.7 Incremental Graph Maintenance
5.8 Temporal Graphs
5.9 GNN Message Passing and Structural Embeddings
6. System Architecture
6.1 Query Pipeline
6.2 SQL Compiler Pipeline
6.3 Physical Execution
6.4 Storage Architecture
6.5 Query Optimizer
6.6 Foreign Data Wrappers and Query Pushdown
6.6.1 Predicate Pushdown
6.6.2 Full Query Pushdown (v0.19.0+)
6.6.3 Mixed Foreign-Local Query Optimization
6.6.4 Arrow Flight SQL Pushdown
6.6.5 Automatic Fallback
7. Theory-to-Code Mapping
Paper 1: Unified Mathematical Framework
Paper 2: Graph Extension
Paper 3: Bayesian BM25
Paper 4: Advanced Fusion and Retrieval
8. Cross-Paradigm Composition
References

---

1. Posting Lists as Universal Abstraction

1.1 The Core Thesis

UQA's foundational claim is that a single data structure — the **posting list** — can serve as the universal abstraction for result sets across relational, text retrieval, vector search, and graph query paradigms.

Definition. A posting list L is an ordered sequence of entries:

$$L = [(id_1, payload_1),\ (id_2, payload_2),\ \ldots,\ (id_n, payload_n)] \tag{1}$$

where $id_i < id_j$ for all $i < j$ (sorted by document ID), and each payload carries positions, scores, and field values.

A bijection $PL : 2^{\mathcal{D}} \to \mathcal{L}$ maps document sets to posting lists and back, allowing set-theoretic reasoning to carry over directly.

1.2 Boolean Algebra

The structure $(\mathcal{L},\ \cup,\ \cap,\ \overline{\cdot},\ \emptyset,\ \mathcal{D})$ forms a **complete Boolean algebra** satisfying commutativity, associativity, distributivity, identity, and complement laws.

Law	Equation
Commutativity	$A \cup B = B \cup A$, $A \cap B = B \cap A$
Associativity	$(A \cup B) \cup C = A \cup (B \cup C)$
Distributivity	$A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$
Identity	$A \cup \emptyset = A$, $A \cap \mathcal{D} = A$
Complement	$A \cup \overline{A} = \mathcal{D}$, $A \cap \overline{A} = \emptyset$

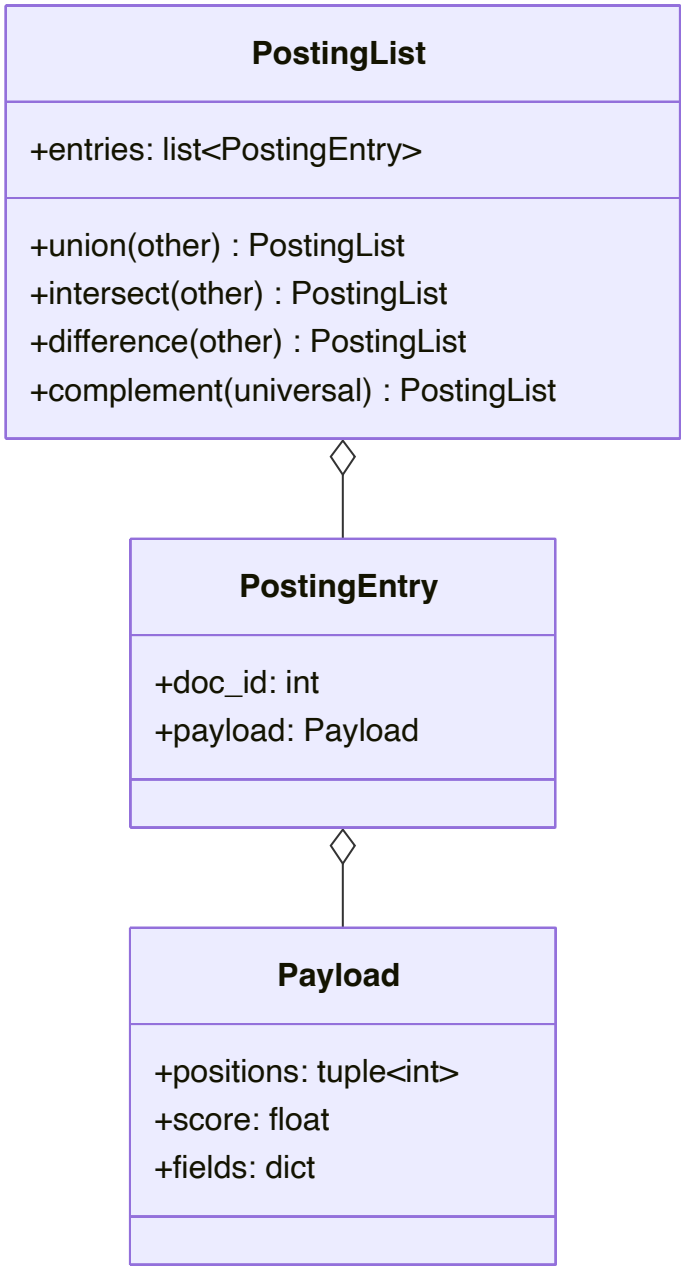
This algebraic closure guarantees that any combination of AND, OR, and NOT across paradigms is well-defined, and query optimization can exploit lattice-theoretic rewrite rules.

1.3 Implementation

The `PostingList` class in `uqa/core/posting_list.py` implements the algebra directly:

```
PostingList
  entries: list[PostingEntry]       # sorted by doc_id
  union(other) -> PostingList       # two-pointer merge, all doc_ids
  intersect(other) -> PostingList   # two-pointer merge, shared doc_ids
  difference(other) -> PostingList  # entries in self but not other
  complement(universal) -> PostingList
```

Operator overloads map to algebra: `|` for union, `&` for intersect, `-` for difference. On merge, payloads are combined: positions are unioned, scores are summed, field dictionaries are merged. The `GeneralizedPostingList` extends this to join results where entries hold tuples of document IDs.



---

2. Cross-Paradigm Operators

2.1 Primitive Operators

Each paradigm maps into the posting list space through a primitive operator:

Operator	Definition	Paradigm	Implementation
$T(t)$	$PL(\{d \in \mathcal{D} \mid t \in term(d, f)\})$	Text retrieval	`TermOperator`
$V_\theta(q)$	$PL(\{d \in \mathcal{D} \mid sim(vec(d, f), q) \geq \theta\})$	Vector search	`VectorSimilarityOperator`
$KNN_k(q)$	$PL(D_k)$ where $\|D_k\| = k$, ranked by similarity	Vector search	`KNNOperator`
$Filter_{f,v}(L)$	$L \cap PL(\{d \in \mathcal{D} \mid d.f = v\})$	Relational	`FilterOperator`
$Spatial_r(c)$	$PL(\{d \in \mathcal{D} \mid haversine(d.f, c) \leq r\})$	Geospatial	`SpatialWithinOperator`
$Score_q(L)$	$(L,\ [s_1, \ldots, s_{\|L\|}])$	Scoring	`ScoreOperator`

Because every operator produces a posting list, they compose freely:

$$Hybrid_{t,q,\theta} = T(t) \cap V_\theta(q) \tag{2}$$

2.2 Operator Monoid

All operators form a monoid under composition. The abstract base class `Operator` in `uqa/operators/base.py` exposes:

- `execute(context: ExecutionContext) -> PostingList` — the universal contract
- `compose(other) -> ComposedOperator` — sequential composition (monoid product)
- `cost_estimate(stats) -> float` — for optimizer decisions

The `ExecutionContext` bundles all storage backends: `DocumentStore`, `InvertedIndex`, `GraphStore`, `BlockMaxIndex`, `IndexManager`, `ParallelExecutor`, optional `vector_indexes` (HNSW, created explicitly via `CREATE INDEX ... USING hnsw`), and optional `spatial_indexes` (R*Tree, created via `CREATE INDEX ... USING rtree`).

2.3 Boolean Operators

Operator	Theory	Implementation
Union $A_1 \cup A_2 \cup \ldots \cup A_n$	Boolean OR	`UnionOperator`
Intersection $A_1 \cap A_2 \cap \ldots \cap A_n$	Boolean AND	`IntersectOperator`
Complement $\mathcal{D} \setminus A$	Boolean NOT	`ComplementOperator`

`UnionOperator` and `IntersectOperator` support parallel execution of their operands via `ParallelExecutor` (ThreadPoolExecutor), with results reduced through `PostingList.union` or `PostingList.intersect`.

---

3. Scoring: From BM25 to Bayesian BM25

3.1 Classical BM25

Robertson and Zaragoza derived BM25 from a probabilistic model of term occurrence. The score for a document d given query term t is:

$$BM25(d,t) = IDF(t) \cdot \frac{tf(t,d) \cdot (k_1 + 1)}{tf(t,d) + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{avgdl}\right)} \tag{3}$$

where $IDF(t) = \ln\left(\frac{N - df(t) + 0.5}{df(t) + 0.5} + 1\right)$ is the Robertson-Sparck Jones inverse document frequency.

Implementation. `BM25Scorer` in `uqa/scoring/bm25.py` implements a numerically stable reformulation:

$$score = w - \frac{w}{1 + f \cdot inv_norm} \tag{4}$$

where $w = boost \cdot IDF$ and $inv_norm = 1/(k_1 \cdot ((1-b) + b \cdot dl/avgdl))$.

The upper bound $ub(t) = boost \cdot IDF(t)$ supports safe document pruning in WAND/BMW.

3.2 The Probabilistic Relevance Gap

BM25 scores are not probabilities. They lie in $[0, +\infty)$ with query-dependent magnitudes, making them incompatible with other ranking signals and violating Robertson's own Probability Ranking Principle. For nearly five decades, this gap persisted.

3.3 Bayesian BM25

Bayesian BM25 closes the gap by applying Bayesian inference with a sigmoid likelihood model to transform BM25 scores into calibrated probabilities $P(R = 1 \mid s) \in [0, 1]$.

Sigmoid Posterior. Under a symmetric likelihood assumption with log-linear likelihood ratio:

$$P(R = 1 \mid s) = \sigma(\alpha(s - \beta)) \tag{5}$$

where $\sigma(x) = \frac{1}{1+e^{-x}}$ is the logistic sigmoid.

Three-Term Decomposition. The full posterior decomposes in log-odds space (additive decomposition of Bayes' theorem):

$$\text{logit}(P_{posterior}) = \underbrace{\text{logit}(L(s))}_{\text{likelihood}} + \underbrace{\text{logit}(b_r(f, \hat{n}))}_{\text{doc-dependent prior}} + \underbrace{\text{logit}(p)}_{\text{base rate}} \tag{6}$$

where $L(s) = \sigma(\alpha(s - \beta))$ is the normalized likelihood ratio, $b_r(f, \hat{n})$ is a composite prior incorporating term frequency rarity and document length, and p is the corpus-level base rate. This reduces expected calibration error by 68-77% without requiring relevance labels.

Implementation. `BayesianBM25Scorer` in `uqa/scoring/bayesian_bm25.py` delegates to the `bayesian-bm25` package, which implements the full three-term posterior with progressive hyperparameter estimation.

3.4 Vector Similarity and Calibration

Cosine similarity $\cos\theta \in [-1, 1]$ is linearly calibrated to probability:

$$P_{vec} = \frac{1 + \cos\theta}{2} \tag{7}$$

This mapping is exact for the case where vector similarity represents angular proximity. `VectorScorer` in `uqa/scoring/vector.py` delegates to `bayesian-bm25`'s `cosine_to_probability()` for numerical stability.

Bayesian vector calibration (Paper 5). The linear mapping above is a first-order approximation. Paper 5 presents a principled Bayesian calibration framework that transforms vector similarity scores into calibrated relevance probabilities through a likelihood ratio formulation:

$$\text{logit } P(R = 1 \mid d) = \log \frac{f_R(d)}{f_G(d)} + \text{logit } P(R = 1) \tag{8}$$

where $f_R(d)$ is the local distance density among relevant documents and $f_G(d)$ is the global background density. Both densities are extracted from ANN index statistics (IVF cell populations, HNSW edge distances) at negligible additional cost. This has the same additive structure as the Bayesian BM25 three-term decomposition, establishing a structural identity between lexical and dense retrieval scoring and completing the probabilistic unification of sparse and dense paradigms.

3.5 Calibration Metrics

To evaluate whether Bayesian BM25's posterior probabilities are well-calibrated — that is, whether a predicted $P(R = 1 \mid s) = 0.7$ corresponds to a 70% observed relevance rate — UQA provides three complementary diagnostics.

Expected Calibration Error (ECE). Documents are binned by predicted probability; ECE is the weighted average of per-bin $|avg(P_i) - avg(y_i)|$. Perfect calibration yields ECE = 0.

Brier Score. The mean squared error $\frac{1}{N}\sum_i (P_i - y_i)^2$ decomposes into calibration error plus discrimination error, providing a single-number summary of probabilistic prediction quality.

Reliability Diagram. A list of $(avg_predicted, avg_actual, count)$ tuples per bin. On a perfectly calibrated system, the points lie on the diagonal $y = x$.

Implementation. `CalibrationMetrics` in `uqa/scoring/calibration.py` wraps the `bayesian-bm25` package's `expected_calibration_error()`, `brier_score()`, `reliability_diagram()`, and `calibration_report()` functions into a unified API.

3.6 Online Parameter Learning

The sigmoid posterior's calibration parameters (α, β, p) can be learned from relevance judgments rather than set manually. `ParameterLearner` in `uqa/scoring/parameter_learner.py` supports two modes:

Batch learning. Given a collection of (score, label) pairs, `fit()` optimizes $(\alpha, \beta, base_rate)$ via the `bayesian-bm25` package's `BayesianProbabilityTransform.fit()`. Optional per-document features (term frequency, document length ratio) enable the three-term decomposition to be jointly calibrated.

Online learning. `update()` processes a single (score, label) observation with stochastic gradient descent at a configurable learning rate, allowing the system to adapt to evolving relevance distributions without full re-training.

3.7 External Prior Scoring

Bayesian BM25's posterior naturally decomposes into likelihood and prior. `ExternalPriorScorer` in `uqa/scoring/external_prior.py` exploits this decomposition by injecting document-level prior knowledge through a user-supplied function $\pi : \text{fields} \to (0, 1)$:

$$\text{logit}(P_{posterior}) = \text{logit}(P_{likelihood}) + \text{logit}(\pi(d)) \tag{9}$$

Two built-in prior factories are provided:

Prior	Function	Formula
Recency	`recency_prior(field, decay_days)`	$\pi = 0.5 + 0.4 \cdot e^{-age/decay}$
Authority	`authority_prior(field, levels)`	Categorical mapping (e.g., high=0.8, medium=0.6, low=0.4)

Both return 0.5 (the neutral prior) when the field is absent, ensuring that missing metadata does not bias scoring.

3.8 Multi-Field Bayesian BM25

When documents contain multiple text fields (title, body, abstract), each field may require independent calibration parameters. `MultiFieldBayesianScorer` in `uqa/scoring/multi_field.py` assigns per-field (α_f, β_f, p_f) parameters and a weight w_f, then combines per-field posteriors via weighted log-odds conjunction:

$$P_{final} = \sigma\left(\sum_f \bar{w}_f \cdot \text{logit}(P_f)\right) \tag{10}$$

where $\bar{w}_f = w_f / \sum_j w_j$ are normalized weights. Fields with zero term frequency contribute a neutral 0.5 posterior, ensuring they neither help nor hurt the fused score.

3.9 Sparse Thresholding (ReLU-as-MAP)

`SparseThresholdOperator` in `uqa/operators/sparse.py` implements a MAP estimation interpretation of ReLU activation as a scoring operator. Given a threshold τ, each document's score is transformed as:

$$s'(d) = \max(0,\ s(d) - \tau) \tag{11}$$

Documents with $s'(d) = 0$ are excluded from the result posting list entirely. Under a Laplace (sparsity-inducing) prior, this thresholding corresponds to the MAP estimate where documents below the threshold have zero posterior probability. The operator composes with any upstream scoring operator via the standard posting list algebra.

In SQL:

```sql
SELECT title, _score FROM papers
WHERE sparse_threshold(text_match(title, 'attention'), 0.3)
ORDER BY _score DESC;
```

---

4. Fusion: Combining Cross-Paradigm Signals

4.1 The Conjunction Shrinkage Problem

Given n independent calibrated relevance signals, the naive probabilistic conjunction:

$$P_{AND} = \prod_{i=1}^{n} P_i \tag{12}$$

suffers from **conjunction shrinkage**: as n increases, $P_{AND} \to 0$ even when all signals report high relevance. This violates the fundamental intuition that agreement among independent sources should increase confidence.

4.2 Log-Odds Conjunction

The log-odds conjunction resolves this by working in the natural parameter space of the Bernoulli distribution:

$$P_{final} = \sigma\left(\frac{1}{n^{1-\alpha}} \sum_{i=1}^{n} \text{logit}(P_i)\right) \tag{13}$$

where $\text{logit}(p) = \log \frac{p}{1-p}$ and α controls confidence scaling.

Key properties:

Property	Guarantee
Scale neutrality	All $P_i = p \implies P_{final} = p$ regardless of n
Sign preservation	$\text{sgn}(\ell_{adjusted}) = \text{sgn}(\bar{\ell})$
Irrelevance preservation	All $P_i < 0.5 \implies P_{final} < 0.5$ for all n
Agreement amplification	All $P_i > 0.5 \implies P_{final} > \sigma(\bar{\ell})$ for $n \geq 2$

Setting $\alpha = 0.5$ yields the classical $\sqrt{n}$ confidence scaling law, where per-signal weight is $w_i = 1/\sqrt{n}$.

This is mathematically equivalent to the **normalized Logarithmic Opinion Pool** (Log-OP), also known as Hinton's **Product of Experts** (PoE, 2002):

$$\text{logit}(P_{Log-OP}) = \sum_{i=1}^{n} \frac{1}{n} \text{logit}(P_i) = \bar{\ell} \tag{14}$$

Numerical comparison ($n = 2$, $\alpha = 0.5$):

P_{text}	P_{vec}	Product	Log-Odds	Interpretation
0.9	0.9	0.81	0.96	Strong agreement amplified
0.7	0.7	0.49	0.77	Moderate agreement preserved
0.7	0.3	0.21	0.50	Exact neutrality (logits cancel)
0.3	0.3	0.09	0.23	Irrelevance preserved

4.3 Probabilistic Boolean Operations

For explicit Boolean query semantics:

Operation	Formula	Log-space
AND	$P = \prod P_i$	$\log P = \sum \log P_i$
OR	$P = 1 - \prod(1 - P_i)$	$\log(1 - P) = \sum \log(1 - P_i)$
NOT	$P = 1 - P_{signal}$	

All computed in log-space for numerical stability.

4.4 Implementation

Function	Class	File
`fuse_log_odds(...)`	`LogOddsFusion`	`uqa/fusion/log_odds.py`
`fuse_prob_and(...)`	`ProbabilisticBoolean`	`uqa/fusion/boolean.py`
`fuse_prob_or(...)`	`ProbabilisticBoolean`	`uqa/fusion/boolean.py`
`fuse_prob_not(...)`	`ProbabilisticBoolean`	`uqa/fusion/boolean.py`
`fuse_log_odds(..., 'relu')`	`LogOddsFusion(gating='relu')`	`uqa/fusion/log_odds.py`
`fuse_mean(...)`	`LogOddsFusion`	`uqa/fusion/log_odds.py`

`fuse_mean()` computes the scale-neutral log-odds mean without confidence scaling: the arithmetic mean in log-odds space mapped back to probability via sigmoid. If all signals report the same probability p, the output is exactly p regardless of n. This implements the normalized Logarithmic Opinion Pool (Definition 4.1.1, Paper 4).

In SQL:

```
SELECT title, _score FROM papers
WHERE fuse_log_odds(
    text_match(title, 'attention'),
    knn_match(embedding, ARRAY[0.1, 0.2, ...], 5)
) ORDER BY _score DESC;
```

4.5 Attention-Weighted Fusion

When signal reliability varies by query, uniform weights are suboptimal. `AttentionFusion` in `uqa/fusion/attention.py` computes per-signal attention weights from a query feature vector via a learned weight matrix $W \in \mathbb{R}^{n \times d}$:

$$w_i = \text{softmax}(W \cdot \mathbf{q})_i \tag{15}$$

where $\mathbf{q} \in \mathbb{R}^d$ is a query feature vector and n is the number of signals. The weighted probabilities are then fused via weighted log-odds conjunction:

$$P_{final} = \sigma\left(\frac{1}{n^{1-\alpha}} \sum_i w_i \cdot \text{logit}(P_i)\right) \tag{16}$$

`QueryFeatureExtractor` in `uqa/fusion/query_features.py` produces a 6-dimensional feature vector: [mean IDF, max IDF, min IDF, coverage ratio, query length, vocabulary overlap ratio]. Both `fit()` (batch) and `update()` (online) training modes are supported, with full state serialization via `state_dict()` / `load_state_dict()`.

In SQL:

```
SELECT title, _score FROM papers
WHERE fuse_attention(
    text_match(title, 'attention'),
    knn_match(embedding, ARRAY[0.1, 0.2, ...], 5)
) ORDER BY _score DESC;
```

4.6 Learned-Weight Fusion

For systems where query features are unavailable or unnecessary, `LearnedFusion` in `uqa/fusion/learned.py` provides a simpler alternative: learned per-signal weights without query conditioning. The weights are trained via `LearnableLogOddsWeights` from the `bayesian-bm25` package, using the same `fit()` / `update()` interface. This is equivalent to attention fusion with a constant query feature vector, reducing the parameter count from $O(n \cdot d)$ to $O(n)$.

In SQL:

```
SELECT title, _score FROM papers
WHERE fuse_learned(
    text_match(title, 'attention'),
    knn_match(embedding, ARRAY[0.1, 0.2, ...], 5)
) ORDER BY _score DESC;
```

4.7 Multi-Signal WAND Pruning

Top-k retrieval over fused signals faces a scalability challenge: naive evaluation requires scoring every document across all signals before selecting the top k. `FusionWANDScorer` in `uqa/scoring/fusion_wand.py` adapts the WAND pivot mechanism to multi-signal fusion.

Because `log_odds_conjunction` is monotone in each input probability, per-signal upper bounds compose safely: $UB_{fused} = \sigma(\frac{1}{n^{1-\alpha}} \sum_i \text{logit}(UB_i))$. Documents whose fused upper bound falls below the current k-th best score are skipped without evaluation. The algorithm maintains a min-heap of size k and computes per-signal coverage-based defaults for documents missing from a signal's posting list.

4.8 Multi-Stage Retrieval Pipeline

`MultiStageOperator` in `uqa/operators/multi_stage.py` implements depth as recursive Bayesian inference: each stage refines the candidate set from the previous stage through progressively more expensive scoring.

A pipeline is a sequence of $(operator, cutoff)$ pairs. Stage 0 executes its operator fully on the corpus. Each subsequent stage re-scores only the surviving documents from the previous stage. The cutoff can be either an integer (top-k by score) or a float (threshold on score):

$$C_0 = \text{top}_k\big(\text{op}_0(\mathcal{D})\big), \quad C_{i+1} = \text{top}_k\big(\text{op}_{i+1}(C_i)\big) \tag{17}$$

The cost model reflects the cascading reduction: each stage's cost is scaled by the fraction of documents surviving from the previous stage, making the total cost sublinear in corpus size when early stages aggressively prune.

4.9 Fusion Gating Mechanisms

The `log_odds_conjunction` function supports optional gating transformations applied to the fused log-odds before the final sigmoid. Two gating functions are available:

ReLU gating. $g(x) = \max(0, x)$. Suppresses weak negative evidence, ensuring that ambiguous signals (near-neutral log-odds) are clamped to zero rather than pulled toward irrelevance.

Swish gating. $g(x) = x \cdot \sigma(\beta x)$. A smooth approximation to ReLU that preserves gradient flow for differentiable pipelines. The self-gating property (σ applied to the input itself) provides implicit regularization.

In SQL, gating is specified as a trailing string argument:

```sql
SELECT title, _score FROM papers
WHERE fuse_log_odds(
    text_match(title, 'attention'),
    knn_match(embedding, $1, 5),
    'relu'
) ORDER BY _score DESC;

-- Combined with alpha parameter:
WHERE fuse_log_odds(sig1, sig2, 0.8, 'swish')
```

The gating parameter is threaded through `LogOddsFusionOperator`, `FusionWANDScorer`, and preserved by the query optimizer during signal reordering.

4.10 Progressive Fusion

`ProgressiveFusionOperator` in `uqa/operators/progressive_fusion.py` implements cascading multi-stage fusion with WAND pruning. Unlike `MultiStageOperator` which re-scores with independent signals at each stage, progressive fusion accumulates signals across stages:

$$\text{Stage}_i : \text{signals}_{1..i} \xrightarrow{\text{WAND top-}k_i} C_i \tag{18}$$

Each stage introduces new signals, computes a fused score over all accumulated signals using `FusionWANDScorer`, and narrows the candidate set to the top k_i documents. Subsequent stages evaluate new signals only on surviving candidates, making the cost cascade: $\text{cost}_i \propto |C_{i-1}|/N$.

4.11 Deep Learning Training Pipeline

Paper 4 establishes that neural networks emerge from Bayesian inference: convolution is neighborhood averaging under a spatial prior, pooling is information-theoretic compression, and the fully connected layer is Bayesian posterior estimation. This insight enables an analytical training pipeline that replaces iterative backpropagation with closed-form parameter estimation.

4.11.1 Analytical Parameter Estimation

The key observation is that each layer type admits either a closed-form solution or requires no parameters at all:

Layer	Parameters	Estimation Method
`ConvLayer`	Multi-channel 3x3 kernels	Configurable initialization: Kaiming random, orthogonal (QR), Gabor filter bank, or k-means patch dictionary
`PoolLayer`	None	Stateless spatial downsampling
`FlattenLayer`	None	Stateless reshaping
`GlobalPoolLayer`	None	Channel-preserving spatial reduction (avg, max, or avg_max)
`DenseLayer`	Weight matrix W, bias b	Ridge regression (Bayesian posterior)
`SoftmaxLayer`	None	Stateless normalization

Convolution as Bayesian prior. For multi-channel convolution ($n_{channels} > 1$), kernels are initialized via one of four strategies and held fixed. The default Kaiming-He initialization samples from $\mathcal{N}(0, \sqrt{2/fan_{in}})$ -- the extreme learning machine paradigm (Huang et al., 2006). Orthogonal initialization (Saxe et al., 2014) uses QR decomposition to produce maximally diverse filters. Gabor initialization generates structured bandpass filters at multiple orientations and frequencies. K-means initialization (Coates & Ng, 2012) extracts 3x3 patches from training images and clusters them to produce data-dependent kernels. In all cases, the random or structured kernels constitute the prior over spatial filters; the subsequent ridge regression layer computes the posterior.

Global pooling as dimensionality reduction. `GlobalPoolLayer` reduces spatial dimensions to 1x1 while preserving channel structure, offering a lower-dimensional alternative to `FlattenLayer`. For a feature map with C channels and $H \times W$ spatial positions, `flatten()` produces a $C \cdot H \cdot W$ dimensional vector while `global_pool('avg')` produces a C dimensional vector. The `avg_max` variant concatenates average and max pooling for a $2C$ dimensional output, capturing both central tendency and peak activation per channel.

Dense layer as Bayesian posterior. Given feature matrix $X \in \mathbb{R}^{n \times d}$ and one-hot label matrix $Y \in \mathbb{R}^{n \times c}$, the dense layer weights are the ridge regression solution:

$$W = (X^T X + \lambda I)^{-1} X^T Y \tag{19}$$

where λ is the regularization parameter (prior precision in Bayesian linear regression). This is the exact MAP estimate under a Gaussian prior $W \sim \mathcal{N}(0, \lambda^{-1} I)$ with Gaussian likelihood — no gradient descent, no learning rate, no epochs.

Implementation. `train_model()` in `uqa/operators/deep_learn.py` implements the full pipeline. `ridge_solve()` in `uqa/operators/_backend.py` computes the closed-form solution, dispatching to PyTorch `torch.linalg.solve()` on GPU when available and falling back to `numpy.linalg.solve()` otherwise.

4.11.2 Product of Experts (PoE) Ensemble

Rather than training a single end-to-end network, the pipeline trains independent expert heads at each conv+pool stage and combines them via the Product of Experts framework (Theorem 8.3).

Each stage s produces a feature representation X_s after convolution and pooling. An expert head W_s is trained via ridge regression on X_s, yielding per-stage logits $\ell_s = X_s W_s^T + b_s$. At inference, the experts are combined via **accuracy-weighted** logit averaging with a diversity-prior shrinkage correction:

$$\ell_{final} = \sum_{s=1}^{n_{experts}} w_s \cdot \ell_s + \alpha \cdot \log(n_{experts}) \tag{20}$$

where $w_s = a_s / \sum_j a_j$ are normalized weights derived from per-stage training accuracy a_s. Experts that achieve higher accuracy on the training set contribute proportionally more to the final prediction. The **diversity-prior shrinkage** parameter $\alpha = 1/(2\sqrt{n_{experts}})$ replaces the fixed 0.5 default, adapting the correction to the number of experts. The additive $\alpha \cdot \log(n_{experts})$ term in log-odds space compensates for the over-counting of shared evidence across experts — the same correction that resolves conjunction shrinkage in Section 4.1, now applied to neural network ensemble combination.

The final class probabilities are obtained by applying softmax to ℓ_{final}.

4.11.3 SQL Interface

The training and inference pipeline is exposed through SQL aggregate and scalar functions:

Training via aggregate. `deep_learn()` operates as a SELECT-clause aggregate function, consuming all rows (optionally filtered by WHERE) to produce a trained model:

```
SELECT deep_learn(
    'model_name', 'images', 'label', 'embedding', 'spatial',
    conv(1, 8), pool(2), flatten(), dense(10), softmax(),
    'relu', 1.0
) FROM images WHERE split = 'train';
```

The aggregate collects rows, extracts embeddings and labels, runs the analytical training pipeline, and persists the trained model to the catalog.

Inference via scalar function. `deep_predict()` operates as a per-row scalar function, applying a trained model to each row's embedding:

```
SELECT id, deep_predict('model_name', embedding) as prediction
FROM images WHERE split = 'test';
```

Inference via deep_fusion. For integration with the fusion framework, `deep_fusion()` accepts a `model()` reference that loads learned weights from the catalog:

```
SELECT * FROM papers
WHERE deep_fusion(model('classifier', $1))
ORDER BY _score DESC;
```

This reuses the `DeepFusionOperator` infrastructure (Section 4.10) with weights populated from a `TrainedModel` rather than specified inline.

4.11.4 GPU Acceleration

The compute backend in `uqa/operators/_backend.py` provides transparent GPU acceleration with a single upload/download boundary per forward pass.

Device detection. At import time, the backend probes for available accelerators in priority order: CUDA (NVIDIA), MPS (Apple Silicon), CPU. All subsequent tensor operations execute on the detected device.

Single-boundary design. `grid_forward()` uploads the embedding matrix to GPU once, executes the full conv+pool pipeline as fused `torch.nn.functional.conv2d()` and `max_pool2d()` / `avg_pool2d()` calls, and downloads the flattened feature matrix once. This eliminates per-layer CPU-GPU transfers that dominate latency in naive implementations.

Fallback. When PyTorch is unavailable, all operations fall back to equivalent numpy implementations with identical numerical behavior (single-channel only).

Function	GPU Path	CPU Fallback
`grid_forward()`	`F.conv2d` + `F.max_pool2d`	Manual padded convolution + reshape pooling
`ridge_solve()`	`torch.linalg.solve`	`numpy.linalg.solve`
`elastic_net_solve()`	Proximal gradient on device	`numpy` ISTA loop
`batch_dense()`	`X @ W^T + b` on device	`numpy` matmul
`batch_softmax()`	`F.softmax`	Shifted exp with manual normalization

Batched grid_forward. For large datasets, `grid_forward()` processes samples in batches of 2048 to prevent GPU out-of-memory errors. Each batch is uploaded, convolved, pooled, and downloaded independently, then results are concatenated. On Apple Silicon (MPS), tensors with total elements exceeding `INT_MAX` ($2^{31} - 1$) fall back to CPU PyTorch, as the MPS backend does not support tensors that large. The same `INT_MAX` guard applies to `ridge_solve()`.

4.11.5 Self-Attention Layer

The `AttentionSpec` layer implements context-dependent Product of Experts (Theorem 8.3, Paper 4). Self-attention determines how strongly each spatial position's evidence is weighted in the ensemble, providing a learned gating mechanism over the spatial feature map.

Three modes are available, representing different assumptions about the query-key-value projections:

Mode	Q	K	V	Description
`content`	X	X	X	Pure content-based attention — positions attend based on their own feature similarity
`random_qk`	$W_Q X$	$W_K X$	X	Random Q,K projections with V=X (extreme learning machine prior)
`learned_v`	$W_Q X$	$W_K X$	$W_V X$	Random Q,K with supervised V projection via hybrid ridge solve

For multi-head attention ($n_{heads} > 1$), the feature dimension is split across heads, each head computes attention independently, and the results are concatenated. The attention computation is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{21}$$

where d_k is the per-head dimension. For `random_qk` and `learned_v` modes, Q and K projection matrices are sampled from a Kaiming-He distribution $\mathcal{N}(0, \sqrt{2/d_{in}})$ and held fixed. In `learned_v` mode, the V projection is learned via ridge regression to maximize alignment with the target labels.

GPU optimization. The attention layer uses chunked attention computation for memory efficiency, single-upload slicing for Q/K/V projections (avoiding redundant CPU-GPU transfers), and dispatches the V ridge solve through the same GPU-accelerated `ridge_solve()` backend.

Implementation. `AttentionSpec` in `uqa/operators/deep_learn.py` defines the layer specification. The attention forward pass is implemented in the training pipeline and integrated into both `deep_learn()` and `deep_fusion()`.

4.11.6 Neural Network Pruning

Post-training pruning reduces model size and can improve generalization by removing redundant parameters. UQA provides two complementary pruning techniques that can be applied independently or in combination.

Elastic net regularization (`l1_ratio > 0`). Combines L1 (lasso) and L2 (ridge) penalties via proximal gradient descent (ISTA — Iterative Shrinkage-Thresholding Algorithm). The objective is:

$$\min_W \frac{1}{n}\|Y - XW\|^2 + \lambda\left(\rho\|W\|_1 + \frac{1-\rho}{2}\|W\|_2^2\right) \tag{22}$$

where ρ is the `l1_ratio` parameter. The solver is warm-started from the ridge regression solution (the $\rho = 0$ case), which provides a good initial point and reduces the number of ISTA iterations needed for convergence. The L1 penalty drives small weights to exactly zero, producing structurally sparse weight matrices. The soft thresholding (proximal) step is:

$$W \leftarrow \text{sign}(W) \cdot \max(|W| - \tau, 0) \tag{23}$$

where $\tau = \lambda\rho/(n \cdot L)$ and L is the Lipschitz constant of the smooth part of the objective.

Magnitude pruning (`prune_ratio > 0`). After training (with or without elastic net), the smallest weights by absolute magnitude are set to zero. A `prune_ratio` of p zeroes the bottom $p \cdot 100\%$ of weights by percentile. The threshold is computed as `numpy.percentile(|W|, p * 100)`, and all weights below this threshold are set to zero.

Combined pruning. When both `l1_ratio > 0` and `prune_ratio > 0`, elastic net drives weights toward sparsity during optimization, and magnitude pruning provides a hard cutoff afterward. This two-phase approach yields sparser models than either technique alone.

Empirical results. On MNIST (28x28 grayscale digits), 50% magnitude pruning retains approximately 94.68% test accuracy compared to the unpruned baseline of 97.89% — a modest accuracy trade-off for halving the number of active parameters.

Implementation. `elastic_net_solve()` and `magnitude_prune()` in `uqa/operators/_backend.py` implement the two pruning backends. Both are GPU-accelerated when PyTorch is available. The `TrainedModel` dataclass stores `l1_ratio`, `prune_ratio`, and `weight_sparsity` metadata for inspection.

4.11.7 Model Catalog

Trained models are persisted to the `_models` table in the SQLite catalog:

```
_models (
    model_name   TEXT PRIMARY KEY,
    config_json  TEXT NOT NULL
)
```

The `config_json` column stores a JSON-serialized `TrainedModel` dataclass containing all learned parameters: conv kernel data, dense weights and biases, expert head weights, layer specifications, class labels, grid geometry, and training metadata (accuracy, sample count). `Engine.save_model()` writes through to both the in-memory `_models` dictionary and the SQLite catalog; `Engine.load_model()` reads from memory first, falling back to SQLite on cache miss.

Theory-to-code mapping. `train_model()` and `predict()` in `uqa/operators/deep_learn.py` implement the training and inference pipelines respectively. `ridge_solve()` and `grid_forward()` in `uqa/operators/_backend.py` implement the core numerical operations with GPU dispatch.

---

5. Graph Extension

5.1 Graph-Posting List Isomorphism

The second paper extends the framework to graph data by establishing an isomorphism between graph posting lists and standard posting lists.

Graph Posting List. A graph posting list $L_G = [(id_1, G_1), \ldots, (id_n, G_n)]$ maps to standard posting lists via:

$$\Phi(L_G) = PL\left(\bigcup_{i=1}^{n} \phi_{G \to D}(G_i)\right) \tag{24}$$

This isomorphism preserves Boolean operations:

$$\Phi(L_G^1 \cup_G L_G^2) = \Phi(L_G^1) \cup \Phi(L_G^2) \tag{25}$$

$$\Phi(L_G^1 \cap_G L_G^2) = \Phi(L_G^1) \cap \Phi(L_G^2) \tag{26}$$

Implementation. `GraphPostingList` in `uqa/graph/posting_list.py` extends `PostingList`, providing `to_posting_list()` (Φ) and `from_posting_list()` (Φ^{-1}). Subgraph structure is encoded in `Payload.fields` as frozen sets of vertex/edge IDs.

5.2 Graph Operators

Operator	Definition	Complexity	Implementation
$Traverse_{v,l,k}$	BFS from v, edges matching l, up to k hops	$O(\sum_{i=1}^{k} d^i)$	`TraverseOperator`
$GMatch_P$	Subgraph isomorphism via backtracking	$O(\|V_G\|^{\|V_P\|})$ (NP-complete)	`PatternMatchOperator`
RPQ_R	Regular path query via NFA simulation	$O(\|V_G\|^2 \cdot \|R\|)$	`RegularPathQueryOperator`
$VertexAgg_{p,f}$	Aggregate property p with function f	$O(\|L_G\|)$	`VertexAggregationOperator`
$PageRank_{d,iter}$	Power iteration centrality	$O(\|V_G\| \cdot iter)$	`PageRankOperator`
$HITS_{iter}$	Hub/authority mutual reinforcement	$O(\|V_G\| \cdot iter)$	`HITSOperator`
BC	Brandes betweenness centrality	$O(\|V_G\| \cdot \|E_G\|)$	`BetweennessCentralityOperator`
$WRPQ_{R,w,f,p}$	Weighted RPQ with aggregate predicates	$O(\|V_G\|^2 \cdot \|R\|)$	`WeightedPathQueryOperator`

All graph operators accept a configurable `score` parameter (default `DEFAULT_GRAPH_SCORE = 0.9`) that controls the relevance probability assigned to reachable vertices. This replaces the previously hard-coded value and allows callers to tune how graph evidence contributes to log-odds fusion with other paradigm signals.

5.3 Subgraph Isomorphism: Complexity and Mitigations

Pattern matching via subgraph isomorphism ($GMatch_P$) has worst-case complexity $O(\|V_G\|^{\|V_P\|})$ and is NP-complete. This is an inherent property of the problem itself — any system supporting general subgraph pattern matching faces the same theoretical bound. The key question is how to keep the **effective** search space tractable.

Naive backtracking iterates all $\|V_G\|$ vertices for each of the $\|V_P\|$ pattern variables in a fixed order, checking edge constraints only after all variables are assigned. For a pattern with k variables on a graph with n vertices, this explores up to n^k complete assignments before discovering that most are invalid.

UQA applies three CSP (Constraint Satisfaction Problem) techniques that reduce the effective search space by orders of magnitude:

1. Candidate Pre-computation with Arc Consistency.
Before backtracking begins, each pattern variable's candidate set is computed by evaluating vertex constraints once against all graph vertices. Then an arc consistency fixpoint loop propagates edge constraints: if variable A must connect to variable B via a labeled edge, any candidate for A that has no such outgoing edge to any candidate of B is eliminated, and vice versa. This propagation repeats until no further reduction occurs.

$$C_v^{(0)} = \{u \in V_G \mid u \text{ satisfies vertex constraints of } v\} \tag{27}$$

$$C_v^{(i+1)} = \{u \in C_v^{(i)} \mid \forall (v, w, l) \in E_P,\ \exists u' \in C_w^{(i)},\ (u, u', l) \in E_G\} \tag{28}$$

This is computed to fixpoint ($C^{(i+1)} = C^{(i)}$), typically converging in 1-2 iterations. The reduction can be dramatic: on a graph with 1M vertices where only 1K have label `Person`, the candidate set shrinks by 1000x before any backtracking occurs.

2. MRV (Minimum Remaining Values) Variable Ordering.
At each recursion step, the algorithm selects the unassigned variable with the fewest remaining candidates — the "fail-first" heuristic. If one variable has 3 candidates and another has 10,000, trying the 3-candidate variable first means at most 3 branches explored before pruning occurs, rather than 10,000.

3. Incremental Edge Validation.
Instead of deferring all edge checks to the end (after all k variables are assigned), each variable binding immediately validates all edge constraints connecting the newly bound variable to previously bound variables. Invalid partial assignments are pruned at depth 2 rather than depth k.

The combined effect:

Optimization	Reduction Factor	Mechanism
Candidate pre-computation	$\|V_G\| \to \|C_v\|$ per variable	Domain initialization
Arc consistency	$\|C_v\| \to \|C'_v\| \leq \|C_v\|$	Edge-based propagation
MRV ordering	Exponential pruning	Fail-first branching
Incremental edge validation	Early cutoff at depth $d \ll k$	Forward checking

In practice, Cypher patterns typically contain 3-7 nodes with label constraints, reducing the search to a small fraction of the graph. The theoretical $O(n^k)$ worst case manifests only for unlabeled patterns on dense graphs with no distinguishing constraints — a scenario rarely encountered in real workloads.

Implementation. `PatternMatchOperator` in `uqa/graph/operators.py` implements all three techniques. `CypherCompiler` in `uqa/graph/cypher/compiler.py` applies label-based candidate filtering via `vertices_by_label()` and incremental vertex/edge validation during path expansion. Variable-length relationship expansion uses BFS with `deque` for O(1) frontier access.

5.4 Regular Path Expressions

Regular path queries use an algebraic expression language over edge labels:

$$R ::= l \;\mid\; R_1 \cdot R_2 \;\mid\; R_1 | R_2 \;\mid\; R^* \tag{29}$$

The `RegularPathExpr` hierarchy (`Label`, `Concat`, `Alternation`, `KleeneStar`) in `uqa/graph/pattern.py` represents these expressions. The `RegularPathQueryOperator` uses Thompson's construction to build an NFA from the expression, then simulates it on the graph.

Bounded repetition. UQA extends the standard RPQ grammar with bounded repetition: $R\{m, n\}$ matches paths of length between m and n hops. The `BoundedLabel` dataclass in `uqa/graph/pattern.py` represents this extension. NFA construction chains m mandatory copies of the inner NFA followed by $(n - m)$ optional copies with epsilon bypasses.

5.5 openCypher Integration

The `CypherCompiler` in `uqa/graph/cypher/` compiles openCypher queries into graph posting list operations:

Every Cypher clause transforms a **binding table** (represented as a `GraphPostingList`). The `CypherQueryOperator` integrates into the standard operator tree — it is not a bypass but a full posting-list citizen, visible in `EXPLAIN` output.

SQL integration via FROM-clause table functions:

```
SELECT * FROM cypher('social', $$
    MATCH (p:Person)-[:KNOWS]->(f:Person)
    WHERE p.age > 25
    RETURN p.name, f.name
$$) AS (person agtype, friend agtype);
```

Named Graphs as Primary Abstraction (v0.20.0). Named graphs replace the flat global graph store with per-graph partitioned adjacency indexes. The `_GraphPartition` class holds per-named-graph adjacency state (vertex_ids, edge_ids, adj_out, adj_in, label_index, vertex_label_index). Vertices and edges are stored globally; adjacency indexes are per-graph with zero duplication. The `GraphStore` redesign requires a `graph` keyword parameter on all mutations and queries.

Graph lifecycle management is provided via `create_graph()`, `drop_graph()`, `has_graph()`, and `graph_names()`. Graph algebra operations -- `union_graphs()`, `intersect_graphs()`, `difference_graphs()`, `copy_graph()` -- compose named graphs. Per-graph statistics (`degree_distribution`, `label_degree`, `vertex_label_counts`) feed the graph cost model.

`SQLiteGraphStore` persists named graphs via `_graph_catalog` and `_graph_membership` tables, ensuring named graphs survive close/reopen cycles.

SQL graph functions now accept direct graph names without the `graph:` prefix (backward compatible):

```
SELECT * FROM traverse(1, 'knows', 2, 'social');
SELECT * FROM rpq('knows/works_with', 1, 'social');
SELECT * FROM temporal_traverse(1, 'knows', 2, 150, 'net');
SELECT * FROM pagerank(0.85, 'social');
SELECT * FROM hits(20, 'social');
SELECT * FROM betweenness('social');
```

5.6 Path Index Integration

Regular path query evaluation via NFA simulation has complexity $O(|V_G|^2 \cdot |R|)$, which can be prohibitive for frequently executed queries. The `PathIndex` in `uqa/graph/index.py` pre-computes reachable $(start, end)$ vertex pairs for specific label sequences, reducing RPQ evaluation for indexed paths to $O(1)$ lookup.

Given a label sequence $[l_1, l_2, \ldots, l_k]$, the path index stores all vertex pairs (v_s, v_e) such that there exists a path $v_s \xrightarrow{l_1} \cdots \xrightarrow{l_k} v_e$ in the graph. The `lookup()` method returns the precomputed set directly, bypassing NFA simulation entirely.

Three index types are available:

Index	Purpose	Lookup
`LabelIndex`	Edge label to edge/vertex ID mapping	$O(1)$ per label
`NeighborhoodIndex`	k-hop neighborhood cache per vertex	$O(1)$ per (v, k)
`PathIndex`	$(start, end)$ pairs for label sequences	$O(1)$ per sequence

The `VersionedGraphStore` (Section 5.7) integrates with path indexes by tracking affected edge labels on mutation, enabling targeted invalidation of stale index entries.

5.6.1 Subgraph Indexing

`SubgraphIndex` in `uqa/graph/index.py` pre-indexes frequently queried subgraph patterns. Given a list of `GraphPattern` objects, `build()` runs `PatternMatchOperator` for each and caches the resulting vertex sets keyed by a canonical string representation. Subsequent `lookup()` calls return cached results in O(1).

Method	Description
`build(graph, patterns)`	Run pattern matching, cache results
`lookup(pattern)`	O(1) cached lookup, returns `set[frozenset[int]]` or `None`
`has_pattern(pattern)`	Check if pattern is indexed
`invalidate(labels)`	Remove entries with matching edge labels

`PatternMatchOperator.execute()` checks `ctx.subgraph_index` before backtracking. When a cached result exists, it bypasses the full CSP solver entirely.

5.7.1 Incremental Pattern Matching

`IncrementalPatternMatcher` in `uqa/graph/incremental_match.py` maintains a set of pattern matches and updates them efficiently when the graph changes:

1. **Invalidate**: Remove matches containing any affected vertex IDs
2. **Re-match**: For each pattern variable, constrain it to affected vertices and run `PatternMatchOperator`
3. **Merge**: Union new matches into the base set

This avoids full re-computation when only a few vertices or edges change.

5.7 Incremental Graph Maintenance

Graph mutations — adding or removing vertices and edges — require careful coordination with indexes and query state. UQA provides two mechanisms for incremental maintenance.

GraphDelta. A `GraphDelta` in `uqa/graph/delta.py` records a sequence of mutation operations (add/remove vertex/edge) that can be applied atomically. It tracks affected vertex IDs and edge labels, enabling targeted index invalidation rather than full rebuild.

VersionedGraphStore. `VersionedGraphStore` in `uqa/graph/versioned_store.py` wraps a base `GraphStore` with version tracking and rollback support. Each `apply(delta)` increments the version counter and stores the inverse delta for undo:

Operation	Inverse
`add_vertex(v)`	`remove_vertex(v.id)`
`remove_vertex(id)`	`add_vertex(saved_v)`
`add_edge(e)`	`remove_edge(e.id)`
`remove_edge(id)`	`add_edge(saved_e)`

`rollback(to_version)` reverse-applies stored inverse deltas until the target version is reached. Invalidation callbacks notify registered path indexes when mutations affect indexed label sequences, ensuring stale entries are purged.

5.8 Temporal Graphs

Many graph datasets carry temporal semantics: edges represent relationships that are valid only during specific time intervals. UQA supports temporal graph queries through `valid_from` / `valid_to` edge properties and a `TemporalFilter` in `uqa/graph/temporal_filter.py`.

Point-in-time query. Given a timestamp t, an edge is valid if $valid_from \leq t \leq valid_to$. This answers "what was the state of the graph at time t?"

Range query. Given an interval $[t_s, t_e]$, an edge is valid if its validity interval overlaps: $valid_from \leq t_e \wedge valid_to \geq t_s$. This answers "which relationships were active at any point during the window?"

Edges without temporal properties are always valid, ensuring backward compatibility with non-temporal graphs. The temporal filter integrates with both traversal and pattern matching:

Operator	File
`TemporalTraverseOperator`	`uqa/graph/temporal_traverse.py`
`TemporalPatternMatchOperator`	`uqa/graph/temporal_pattern_match.py`

Both operators push the temporal predicate into edge evaluation, pruning invalid edges before they contribute to traversal frontiers or pattern matches.

5.9 GNN Message Passing and Structural Embeddings

UQA extends the graph-posting list framework with two operators that bridge graph structure and scoring.

Message Passing. `MessagePassingOperator` in `uqa/graph/message_passing.py` performs k rounds of neighbor feature aggregation with a residual connection:

$$h_v^{(l+1)} = h_v^{(l)} + \text{agg}\Big(\{h_u^{(l)} \mid u \in \mathcal{N}(v)\}\Big) \tag{30}$$

where $\text{agg} \in \{\text{mean}, \text{sum}, \text{max}\}$ is configurable. After k layers, features are calibrated via sigmoid to produce per-vertex relevance probabilities $P(R = 1 \mid v) = \sigma(h_v^{(k)})$, yielding a `GraphPostingList` that composes with all other operators.

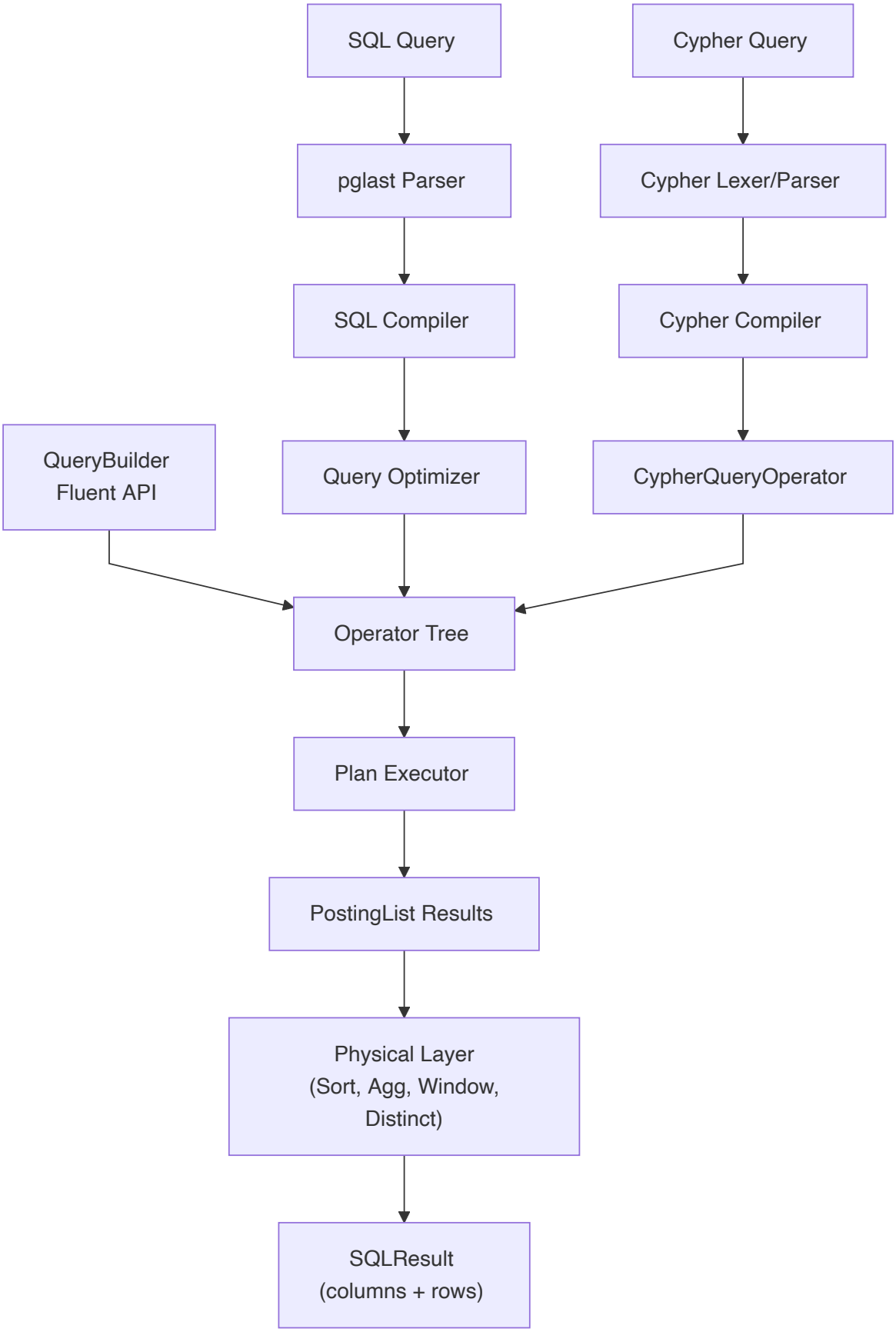
Structural Embeddings. `GraphEmbeddingOperator` in `uqa/graph/graph_embedding.py` computes fixed-dimensional embeddings from graph structure alone — degree (in/out), neighbor label distribution, and k-hop connectivity statistics. Embeddings are L2-normalized and stored in the payload's `_embedding` field, enabling downstream vector similarity search over graph structure without requiring external feature vectors.

---

6. System Architecture

6.1 Query Pipeline

All queries — regardless of entry point (SQL, Cypher, Fluent API) — produce intermediate and final results as posting lists:



6.2 SQL Compiler Pipeline

The SQL compiler in `uqa/sql/compiler.py` follows a staged pipeline:

1. **Parse.** pglast (PostgreSQL parser) produces an AST.

2. **Route.** Statement type dispatch (SELECT, INSERT, UPDATE, DELETE, DDL, utility).
3. **Compile SELECT:**
    - Materialize CTEs as temporary tables
    - Resolve FROM clause to (Table, source_op)
    - Create `ExecutionContext` from table storage
    - Process WHERE clause into operator tree
    - Optimize via `QueryOptimizer`
    - Execute operator tree to `PostingList`
    - Apply relational layer (GROUP BY, HAVING, ORDER BY, LIMIT, DISTINCT) via physical operators

6.3 Physical Execution

The physical layer in `uqa/execution/` uses the **Volcano iterator model**: each `PhysicalOperator` implements `open() / next() / close()`, pulling batches from children on demand (pipelined execution).

Operator	Type	Strategy
`FilterOp`	Streaming	Vectorized Arrow compute for simple predicates; per-row fallback for LIKE, BETWEEN
`ExprFilterOp`	Streaming	Complex expressions via `ExprEvaluator`
`ProjectOp`	Streaming	Column selection
`ExprProjectOp`	Streaming	Vectorized Arrow compute for column refs and arithmetic; per-row fallback for complex expressions
`SortOp`	Blocking	Arrow-native `pc.sort_indices()` with Python stable sort fallback; external merge sort for disk spilling
`HashAggOp`	Blocking	Grace hash partitioning (spills to disk)
`DistinctOp`	Blocking	Hash partition dedup
`LimitOp`	Streaming	LIMIT/OFFSET
`WindowOp`	Blocking	Window function evaluation with bisect-based RANGE frame bounds

6.4 Storage Architecture

Each SQL table maintains isolated storage instances:

```
Engine
+-- _tables: dict[str, Table]
|   +-- Table
|       +-- DocumentStore         (typed columnar storage)
|       +-- InvertedIndex         (term -> posting list per field)
```

```
|       +-- vector_indexes         (HNSW, created via CREATE INDEX ... USING hnsw)
|       +-- spatial_indexes        (R*Tree, created via CREATE INDEX ... USING rtree)
|       +-- ColumnStats           (per-column statistics for optimizer)
+-- _named_graphs: dict[str, GraphStore]
|   +-- GraphStore                (adjacency list + property maps)
+-- _foreign_servers: dict[str, ForeignServer]
+-- _foreign_tables: dict[str, ForeignTable]
+-- _fdw_handlers: dict[str, FDWHandler]
+-- _catalog: Catalog             (SQLite, write-through persistence)
+-- _transaction: Transaction     (BEGIN/COMMIT/ROLLBACK/SAVEPOINT)
```

When `db_path` is provided, all mutations are write-through to SQLite. On startup, `_load_from_catalog()` restores the full state.

6.5 Query Optimizer

The cost-based optimizer in `uqa/planner/` applies:

- **DPccp join order optimization** (Moerkotte & Neumann, 2006) — $O(3^n)$ dynamic programming over connected subgraph complement pairs for optimal bushy join trees; uses integer bitmask DP table, bytearray connectivity lookup, incremental connected subgraph enumeration, and canonical submask enumeration for efficient inner-loop execution
- Filter pushdown into intersections
- Vector threshold merge (same query vector)
- Intersect operand reordering by cardinality (cheapest first)
- Fusion signal reordering by cost
- R*Tree spatial index scan for POINT column range queries
- B-tree index scan substitution
- FDW predicate pushdown (comparison, IN, LIKE, ILIKE, BETWEEN) and full query pushdown for same-server foreign tables (see Section 6.6)
- **EXISTS subquery decorrelation** — correlated `EXISTS` subqueries with equijoin predicates are decorrelated into semi-join `FilterOperator(outer_col, InSet(values))`, executing the inner query once instead of once per outer row (116x speedup on the EXISTS benchmark)
- Edge property filter pushdown into graph pattern constraints (e.g., `"a_b.since" > 2020` pushed into `EdgePattern.constraints`)
- Join-pattern fusion: merge intersected `PatternMatchOperator` children with shared vertex variables into single pattern
- **Algebraic simplification** (Theorem 6.1.2, Paper 1) — idempotent intersection/union ($A \cap A = A$, $A \cup A = A$), absorption law ($A \cap (A \cup B) = A$), and empty elimination ($A \cap \emptyset = \emptyset$, $A \cup \emptyset = A$) are applied bottom-up before other rewrites, reducing operator tree size before cost estimation
- Cross-paradigm join cost models for text similarity, vector similarity, graph, hybrid, and cross-paradigm joins

- Threshold-aware vector selectivity estimation (4-tier threshold buckets: 0.9/0.7/0.5/<0.5)
- Temporal graph cardinality correction with timestamp/range selectivity
- Random-walk graph sampling for cardinality estimation on graphs with 10000+ vertices

Cardinality estimation uses equi-depth histograms and Most Common Values (MCV), with specialized estimators for text, vector, graph, and fusion operators. The `_column_entropy()` function estimates per-column entropy with a three-tier priority: (1) MCV frequencies when available, (2) histogram bucket frequencies for equi-depth histograms, (3) uniform assumption over the distinct count as a fallback.

6.6 Foreign Data Wrappers and Query Pushdown

UQA implements the PostgreSQL Foreign Data Wrapper (FDW) pattern to federate queries across external data sources. Two built-in handlers ship with the system:

Handler	Backend	Transport	Use Cases
`duckdb_fdw`	DuckDB (in-process)	Direct library call	Parquet, CSV, S3, attached databases
`arrow_fdw`	Arrow Flight SQL	gRPC	Dremio, DataFusion, remote OLAP engines

Foreign tables are declared via standard SQL DDL:

```sql
CREATE SERVER warehouse FOREIGN DATA WRAPPER duckdb_fdw;
CREATE FOREIGN TABLE events (
    id INTEGER,
    ts TIMESTAMP,
    payload TEXT
) SERVER warehouse OPTIONS (source 's3://bucket/events/*.parquet');
```

6.6.1 Predicate Pushdown

For single-table scans, the compiler extracts pushable predicates from the WHERE clause and forwards them to the FDW handler. Supported predicate types: comparison (`=`, `<`, `>`, `<=`, `>=`, `!=`), `IN`, `LIKE`, `ILIKE`, and `BETWEEN`. Predicates that cannot be pushed down are retained as post-scan filters in the physical execution layer. The extraction is handled by `_extract_pushdown_predicates()` in `uqa/sql/compiler.py`, which splits the WHERE AST into pushable and remaining parts.

6.6.2 Full Query Pushdown (v0.19.0+)

When a SELECT statement references only foreign tables (and optionally small local tables) that all reside on the same FDW server, the compiler pushes the **entire query** -- including JOINs, aggregations, window functions, subqueries, ORDER BY, and LIMIT -- to the remote engine. This eliminates PostingEntry materialization in Python entirely.

The pushdown pipeline in `_try_foreign_full_pushdown()` proceeds as follows:

1. **AST table collection.** `_collect_ast_table_refs()` walks the full pglast AST (FROM, JOINs, subqueries, CTEs) to extract every `RangeVar` table reference.

2. **Server affinity check.** Each referenced table is classified as foreign or local. All foreign tables must belong to the same server. If tables span multiple servers or include unknown references (CTEs, views), the pushdown is abandoned.
3. **Source registration.** For DuckDB, each foreign table is registered as a DuckDB view pointing to its source expression (`read_parquet(...)`, etc.). For mixed queries, local tables are materialized as DuckDB temp tables via `_ship_local_table_to_duckdb()`.
4. **AST deparsing.** The original pglast AST is converted back to a SQL string using `pglast.stream.RawStream`. This preserves the full query semantics -- all clauses, expressions, and syntax -- without manual SQL string construction.
5. **Remote execution.** The deparsed SQL is executed on the target engine (DuckDB connection or Flight SQL endpoint), and the resulting Arrow table is converted to `SQLResult` via `_arrow_to_sql_result()`.
6. **Cleanup.** Temporary views and tables created for pushdown are dropped in a `finally` block regardless of success or failure.

Decision logic by handler type:

Scenario	`duckdb_fdw`	`arrow_fdw`
All tables foreign, same server	Full pushdown	Full pushdown
Mixed foreign + local tables	Full pushdown (local shipped to DuckDB)	Falls back to UQA pipeline
Multiple servers	Falls back to UQA pipeline	Falls back to UQA pipeline
EXPLAIN query	Falls back to UQA pipeline (plan tree needed)	Falls back to UQA pipeline

6.6.3 Mixed Foreign-Local Query Optimization

DuckDB's in-process architecture enables a unique optimization: when a query joins foreign tables with local UQA tables, the local tables can be shipped into DuckDB as temporary tables rather than pulling all foreign data into Python. This is beneficial when the local table is small and the foreign data is large.

The `_ship_local_table_to_duckdb()` method reads all rows from the local table's document store, constructs a PyArrow table, and registers it as a DuckDB temp table. A safety threshold of 100,000 rows (`_MAX_LOCAL_ROWS`) prevents shipping large local tables. If any local table exceeds this limit, the entire pushdown is abandoned and the query falls back to the standard UQA operator pipeline.

POINT columns receive special handling: since DuckDB lacks UQA's POINT type, each POINT column is expanded into two DOUBLE columns (`col_lon`, `col_lat`).

6.6.4 Arrow Flight SQL Pushdown

For `arrow_fdw` servers, full query pushdown is limited to pure-foreign queries (no local table mixing). The deparsed SQL undergoes table name substitution -- each foreign table name is replaced with its source expression (a remote table name or a subquery from the `query` option). The rewritten SQL is sent to the Flight SQL server via `get_flight_info()` / `do_get()`, and the returned Arrow record batches are concatenated into the final result.

6.6.5 Automatic Fallback

Every pushdown attempt is wrapped in exception handling. If any step fails -- source registration, SQL deparsing, remote execution, or result conversion -- the method returns `None`, and the compiler falls back to the standard UQA operator pipeline with per-predicate pushdown. This design ensures that unsupported SQL constructs, network failures, or DuckDB/Flight SQL incompatibilities never cause query failures. The fallback path treats foreign tables as `_ForeignTableScanOperator` nodes with individual predicate pushdown, which is always correct albeit slower.

The pushdown check runs as step 1 in `_compile_select()`, before FROM resolution or WHERE processing. EXPLAIN queries skip pushdown entirely so that the UQA plan tree is available for inspection.

---

7. Theory-to-Code Mapping

The table below maps each formal definition and theorem from the four papers to its concrete implementation:

Paper 1: Unified Mathematical Framework

Theory	Implementation
Posting List (Definition 2.1.1)	`PostingList` in `uqa/core/posting_list.py`
Boolean Algebra (Theorem 2.1.2)	`union()`, `intersect()`, `difference()`, `complement()`
Generalized Posting List (Definition 4.1.2)	`GeneralizedPostingList` in `uqa/core/posting_list.py`
Operator Monoid (Theorem 3.2.3)	`Operator` ABC + `compose()` in `uqa/operators/base.py`
Term Operator $T(t)$ (Definition 3.1.1)	`TermOperator` in `uqa/operators/primitive.py`
Vector Operators V_θ, KNN_k (Definitions 3.1.2, 3.1.3)	`VectorSimilarityOperator`, `KNNOperator`
Spatial Operator $Spatial_r(c)$	`SpatialWithinOperator` in `uqa/operators/primitive.py`
Filter Operator (Definition 3.1.4)	`FilterOperator` in `uqa/operators/primitive.py`
Score Operator (Definition 3.1.5)	`ScoreOperator` in `uqa/operators/primitive.py`
Hierarchical Documents (Definition 5.2.1)	`HierarchicalDocument` in `uqa/core/hierarchical.py`
Hierarchical Operators (Definitions 5.3.1-5.3.5)	`PathFilterOperator`, `PathProjectOperator`, `PathUnnestOperator`, `PathAggregateOperator`, `UnifiedFilterOperator` in `uqa/operators/hierarchical.py` — all five implement `cost_estimate()` for optimizer integration
Aggregation Monoids (Definition 4.2.1)	`CountMonoid`, `SumMonoid`, `AvgMonoid`, `MinMonoid`, `MaxMonoid`, `QuantileMonoid` in `uqa/operators/aggregation.py`
BM25 (Definition 3.2.1)	`BM25Scorer` in `uqa/scoring/bm25.py`
UnionOperator	`UnionOperator` in `uqa/operators/boolean.py`
IntersectOperator	`IntersectOperator` in `uqa/operators/boolean.py`
ComplementOperator	`ComplementOperator` in `uqa/operators/boolean.py`

Paper 2: Graph Extension

Theory	Implementation

Graph-Posting List Isomorphism (Theorem 1.1.6)	`GraphPostingList` in `uqa/graph/posting_list.py`
Traversal Operator (Definition 2.2.1)	`TraverseOperator` in `uqa/graph/operators.py`
Pattern Match (Definition 5.2.2)	`PatternMatchOperator` in `uqa/graph/operators.py`
Regular Path Query (Definition 5.1.2)	`RegularPathQueryOperator` in `uqa/graph/operators.py`
Vertex Aggregation (Definition 2.2.3)	`VertexAggregationOperator` in `uqa/graph/operators.py`
Graph Pattern Expression (Definition 5.2.1)	`GraphPattern` in `uqa/graph/pattern.py`
Regular Path Expression (Definition 5.1.1)	`RegularPathExpr` hierarchy in `uqa/graph/pattern.py`
openCypher (cross-paradigm integration)	`CypherCompiler` in `uqa/graph/cypher/`
Path Index (Section 6.4)	`PathIndex` in `uqa/graph/index.py`
Label Index (Section 6.4)	`LabelIndex` in `uqa/graph/index.py`
Neighborhood Index (Section 6.4)	`NeighborhoodIndex` in `uqa/graph/index.py`
GraphDelta (Section 9.3)	`GraphDelta` in `uqa/graph/delta.py`
VersionedGraphStore (Section 9.3)	`VersionedGraphStore` in `uqa/graph/versioned_store.py`
Temporal Graph Filter (Section 10)	`TemporalFilter` in `uqa/graph/temporal_filter.py`
Temporal Traversal (Section 10)	`TemporalTraverseOperator` in `uqa/graph/temporal_traverse.py`
Temporal Pattern Match (Section 10)	`TemporalPatternMatchOperator` in `uqa/graph/temporal_pattern_match.py`
Graph Centrality (Section 9.1)	`PageRankOperator`, `HITSOperator`, `BetweennessCentralityOperator` in `uqa/graph/centrality.py`
Bounded RPQ (Section 5.1)	`BoundedLabel` in `uqa/graph/pattern.py`
Weighted RPQ (Section 5.1)	`WeightedPathQueryOperator` in `uqa/graph/operators.py`
Subgraph Index (Section 9.1 #4)	`SubgraphIndex` in `uqa/graph/index.py`
Incremental Pattern Matching (Section 9.3)	`IncrementalPatternMatcher` in `uqa/graph/incremental_match.py`
Graph Sampling (Section 6.3)	`_sample_graph_cardinality()` in `uqa/planner/cardinality.py`
Edge Filter Pushdown (Theorem 6.1.1)	`_push_graph_pattern_filters()` edge case in `uqa/planner/optimizer.py`

Join-Pattern Fusion (Theorem 6.1.2)	`_fuse_join_pattern()` + `_merge_patterns()` in `uqa/planner/optimizer.py`
Temporal Cardinality (Section 8)	`_temporal_selectivity()` in `uqa/planner/cardinality.py`

Paper 3: Bayesian BM25

Theory	Implementation
Sigmoid Posterior (Theorem 4.1.1)	`BayesianBM25Scorer.score()` in `uqa/scoring/bayesian_bm25.py`
Three-Term Decomposition (Theorem 4.4.2)	Two successive Bayes updates in `bayesian-bm25` package
Vector Calibration (Definition 7.1.2)	`VectorScorer.similarity_to_probability()` in `uqa/scoring/vector.py`
Probabilistic AND (Theorem 5.1.1)	`ProbabilisticBoolean.prob_and()` in `uqa/fusion/boolean.py`
Probabilistic OR (Theorem 5.2.1)	`ProbabilisticBoolean.prob_or()` in `uqa/fusion/boolean.py`
WAND Upper Bound (Theorem 6.1.2)	`BayesianBM25Scorer.upper_bound()`
Calibration Metrics (Section 11.3)	`CalibrationMetrics` in `uqa/scoring/calibration.py`
Online Parameter Learning (Section 8)	`ParameterLearner` in `uqa/scoring/parameter_learner.py`
External Prior Scoring (Section 12.2 #6)	`ExternalPriorScorer` in `uqa/scoring/external_prior.py`
Multi-Field Bayesian BM25 (Section 12.2 #1)	`MultiFieldBayesianScorer` in `uqa/scoring/multi_field.py`

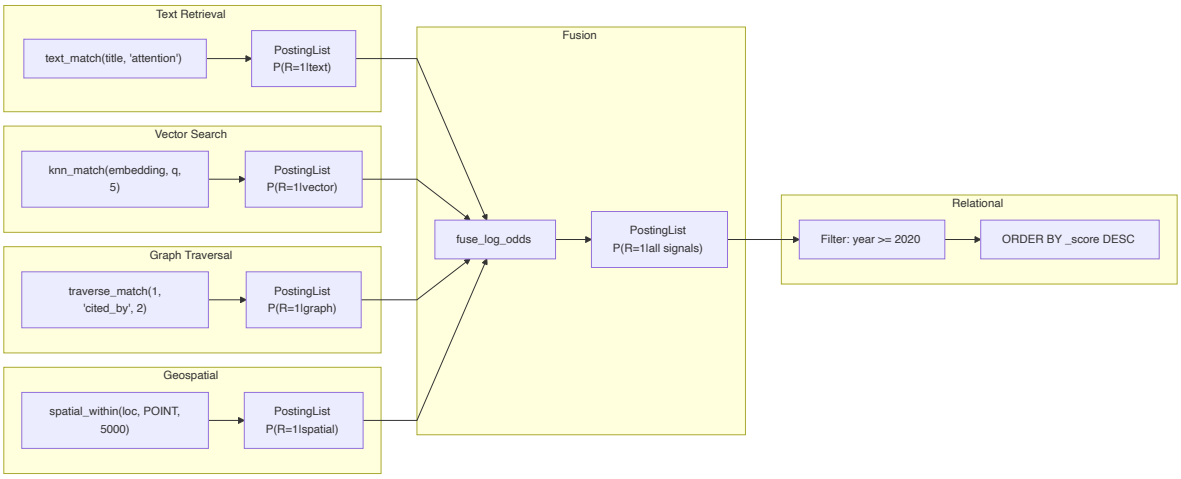
Paper 4: Advanced Fusion and Retrieval

Theory	Implementation
Sparse Thresholding / ReLU-as-MAP (Section 6.5)	`SparseThresholdOperator` in `uqa/operators/sparse.py`
Attention-Weighted Fusion (Section 8)	`AttentionFusion` in `uqa/fusion/attention.py`
Query Feature Extraction (Section 8)	`QueryFeatureExtractor` in `uqa/fusion/query_features.py`
Learned-Weight Fusion (Section 8)	`LearnedFusion` in `uqa/fusion/learned.py`
Multi-Signal WAND Pruning	

(Section 8.7)	`FusionWANDScorer` in `uqa/scoring/fusion_wand.py`
Multi-Stage Retrieval Pipeline (Section 9)	`MultiStageOperator` in `uqa/operators/multi_stage.py`
GNN Message Passing (Paper 2 + Paper 4)	`MessagePassingOperator` in `uqa/graph/message_passing.py`
Structural Graph Embeddings (Paper 2 + Paper 4)	`GraphEmbeddingOperator` in `uqa/graph/graph_embedding.py`
Fusion Gating (Section 6.5-6.7)	`gating` parameter in `LogOddsFusion`, `LogOddsFusionOperator`, `FusionWANDScorer`
Progressive Fusion (Section 7)	`ProgressiveFusionOperator` in `uqa/operators/progressive_fusion.py`
Analytical Training / PoE (Theorem 8.3)	`train_model()`, `predict()` in `uqa/operators/deep_learn.py`
Self-Attention Layer (Theorem 8.3)	`AttentionSpec` in `uqa/operators/deep_learn.py`
Accuracy-Weighted PoE	Per-stage accuracy weighting in `train_model()` and `predict()`
Diversity-Prior Shrinkage	$\alpha = 1/(2\sqrt{n_{experts}})$ in `train_model()`
Ridge Regression Posterior	`ridge_solve()` in `uqa/operators/_backend.py`
Elastic Net (Proximal GD / ISTA)	`elastic_net_solve()` in `uqa/operators/_backend.py`
Magnitude Pruning	`magnitude_prune()` in `uqa/operators/_backend.py`
GPU-Accelerated Forward Pass	`grid_forward()` in `uqa/operators/_backend.py`
Batched Grid Forward (2048 samples)	`grid_forward()` batch loop in `uqa/operators/_backend.py`
Model Persistence	`_models` table in `uqa/storage/catalog.py`; `save_model()` / `load_model()` in `uqa/engine.py`
Cross-Paradigm Join Costs (Section 4)	Cost/cardinality models in `uqa/planner/cost_model.py`, `uqa/planner/cardinality.py`
Vector Selectivity (Section 5.3)	`_vector_selectivity()` in `uqa/planner/cardinality.py`

8. Cross-Paradigm Composition

The key invariant — **every query produces a PostingList** — enables seamless cross-paradigm composition:



In SQL, this is a single query:

```sql
SELECT title, _score FROM papers
WHERE fuse_log_odds(
    text_match(title, 'attention'),
    knn_match(embedding, ARRAY[0.1, 0.2, ...], 5),
    traverse_match(1, 'cited_by', 2)
) AND year >= 2020
ORDER BY _score DESC;
```

The query optimizer ensures efficient execution: each paradigm's operator executes independently (potentially in parallel), their posting lists are intersected via the fusion operator, and the relational filter is applied on the fused result.

References

1. Jeong, J. (2023). A Unified Mathematical Framework for Query Algebras Across Heterogeneous Data Paradigms.
2. Jeong, J. (2024). Extending the Unified Mathematical Framework to Support Graph Data Structures.
3. Jeong, J. (2026). Bayesian BM25: A Probabilistic Framework for Hybrid Text and Vector Search.
4. Jeong, J. (2026). Advanced Fusion and Retrieval: Attention-Weighted Log-Odds, Multi-Stage Pipelines, and GNN Integration.
5. Jeong, J. (2026). Vector Scores as Likelihood Ratios: Index-Derived Bayesian Calibration for Hybrid Search.